- The `billing` service gathers all the payment info and calls out to a third-party provider to handle the payment.
- Last but not least, if everything is in order, the checkout service invokes the `inventory` service to update what is in stock.

The representation of the microservices in figure 8.2 has different names in observability frontends, such as *service map* or *dependency graph*. This service map provides a nice overview of the architecture and shows the flow of information. However, it does not capture the temporal aspects. In other words, we need a way to visualize the service execution along a time axis, typically called a *waterfall visualization*. Figure 8.3 shows one such alternative way to represent the invocations the checkout service does based on time elapsing.

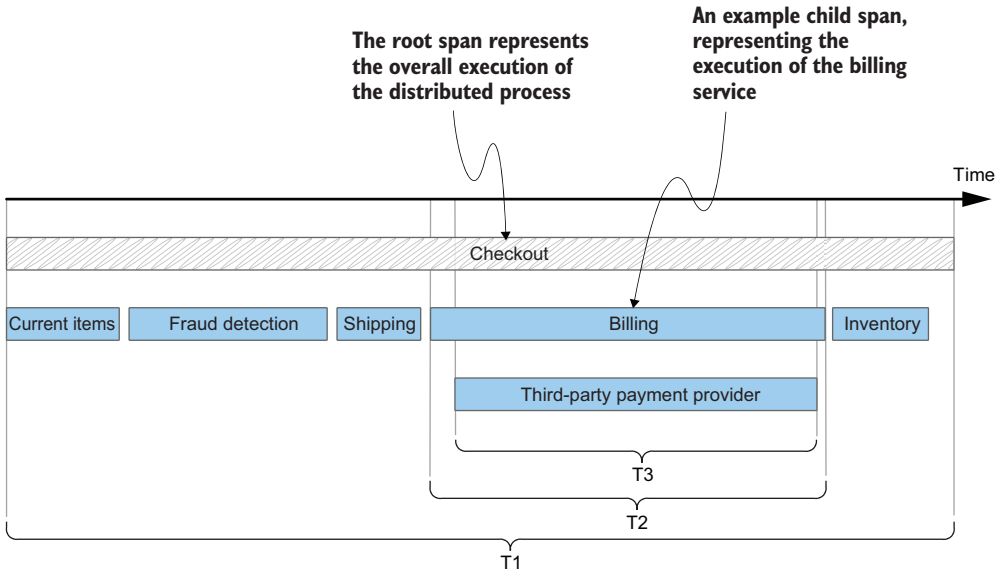


Figure 8.3 A single request path in the app, as a temporal (waterfall) visualization

The entire unit depicted in figure 8.3 is called a *trace* and represents the end-to-end processing of the catalog request in a waterfall visualization. Each service invocation, such as the `checkout` service calling the `billing` service, is a span (compare our discussion in chapter 2):

- The root span is the overall execution of the `checkout` service and takes the time `T1` to execute end to end.
- Examples of child spans are the execution of the `billing` service (which takes `T2` to complete) and the execution of the third-party payment provider call (which takes `T3` to complete). Besides some setup time that our billing service has to do, it seems most of the time is taken up by invoking that external service.